# The texwright agent guide

A practical contract for reliable document generation

Sophia Xe5 Thomas Garren

Version 0.3.4

## 1 Write, validate, repair, inspect

Read the writing guide once, then start from a template. Prefer Markdown for prose with mathematics; use LaTeX for existing sources and JSON when your application already has a structured document tree.

### 1.1 A reproducible first run

From a built repository:

```
node dist/cli.mjs new paper -o work/paper.md
node dist/cli.mjs check work/paper.md --diagnostics json
node dist/cli.mjs repair work/paper.md -o work/paper.pdf
node dist/cli.mjs outline work/paper.md --json
node dist/cli.mjs preview work/paper.md --page 1
```

Create the work folder first if needed. The paper template writes its sample bibliography beside the document.

### 1.2 Interpret results deliberately

The check command runs parsing, numbering, mathematics, references, and citations, but not page layout. The repair command applies safe fixes until none remains and renders when the document has no errors. Review suggestions that are not marked safe. Set a retry budget and stop for review if the same diagnostic repeats.

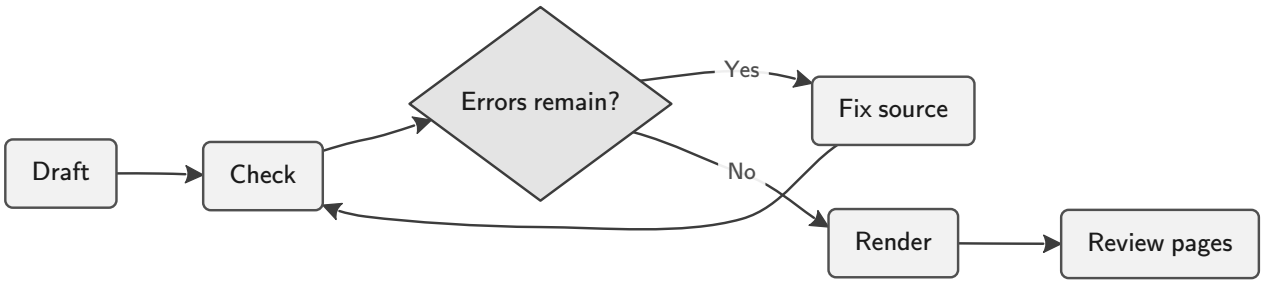



Figure 1: A bounded validation loop.

A best-effort PDF can be written even when rendering fails. Treat the ok field and the CLI exit status as authoritative. Inspect warnings and actual pages before delivering a final document.

## 2 Connect through MCP

Build the repository, then point your MCP client at the local bundle using absolute paths. This JSON illustrates the common mcpServers configuration shape; the enclosing configuration depends on the client.

```
{
  "mcpServers": {
    "texwright": {
      "command": "node",
      "args": [
        "/absolute/path/to/texwright/dist/cli.mjs",
        "mcp", "--root", "/absolute/path/to/documents"
      ]
    }
  }
}
```

On Windows use forward slashes in JSON paths, such as D:/texwright/dist/cli.mjs. The server uses stdio. Keep standard output reserved for the protocol.

Table 1: All nine MCP tools in version 0.3.4.

| Tool | When to use it |
|---|---|
| get_guide | Read supported syntax; short: true gives a short guide |
| new_document | Get a starter document and its companion assets |
| check_document | Validate source without page layout |
| fix_document | Apply safe fixes to an existing file |
| repair_document | Apply safe fixes repeatedly, then render |
| outline_document | Locate headings and numbered objects by source line |
| render_document | Write PDF, HTML, or JSON output |
| preview_page | Inspect selected pages as PNG images |
| get_schema | Obtain the structured document JSON Schema |

The generated MCP tool catalog records the actual names, descriptions, argument schemas, and annotations returned by the built server. Clients should also request tools/list from the running version.

Requests run in worker threads with a configurable time limit. Workspace paths must stay within the configured root; give explicit output paths when your application needs a predictable result location.

# 3 Use exact arguments

The following objects are tool arguments, not a client-specific transport wrapper. For an existing report.md under the server root:

```
{"path":"report.md","compact":true}
```

Pass that object to check_document. To render a PDF, use render_document:

```
{
  "path": "report.md",
  "to": "pdf",
  "output_path": "report.pdf"
}
```

Then inspect the first page with preview_page:

```
{"path":"report.md","pages":[1],"dpi":100}
```

Page numbers are one-based; MCP returns at most four page images per call. For in-memory text, use source and format; set base_dir to resolve accompanying assets. Some tools write source files: fix_document and repair_document require a path.

## 3.1 Diagnose by code

Table 2: Common diagnostics and a useful response.

| Code | Meaning | Next action |
|---|---|---|
| REF001 | Undefined reference | Check the suggested existing label |
| CITE001 | Unknown citation key | Inspect the bibliography |
| MATH001 | Formula parse failure | Read the hint; check braces and macros |
| IMG001 | Missing or disallowed image | Use a file beside the document |
| LAY001 | Content exceeds a line | Shorten or break the content |
| DIAG001 | Diagram could not be parsed | Repair the diagram source |
| TIKZ001 / PST001 | Drawing feature approximated | Review the rendered figure |

## 3.2 Reproducibility and security

Keep source, figures, and bibliography together. Record the output of texwright --version --json to identify the exact build. Set SOURCE_DATE_EPOCH for reproducible PDF timestamps.

Leave raw HTML sanitization enabled for untrusted documents. Keep the MCP root narrow. Use local assets and review any explicit expansion of file or network access. A successful render does not validate the factual content of a document.